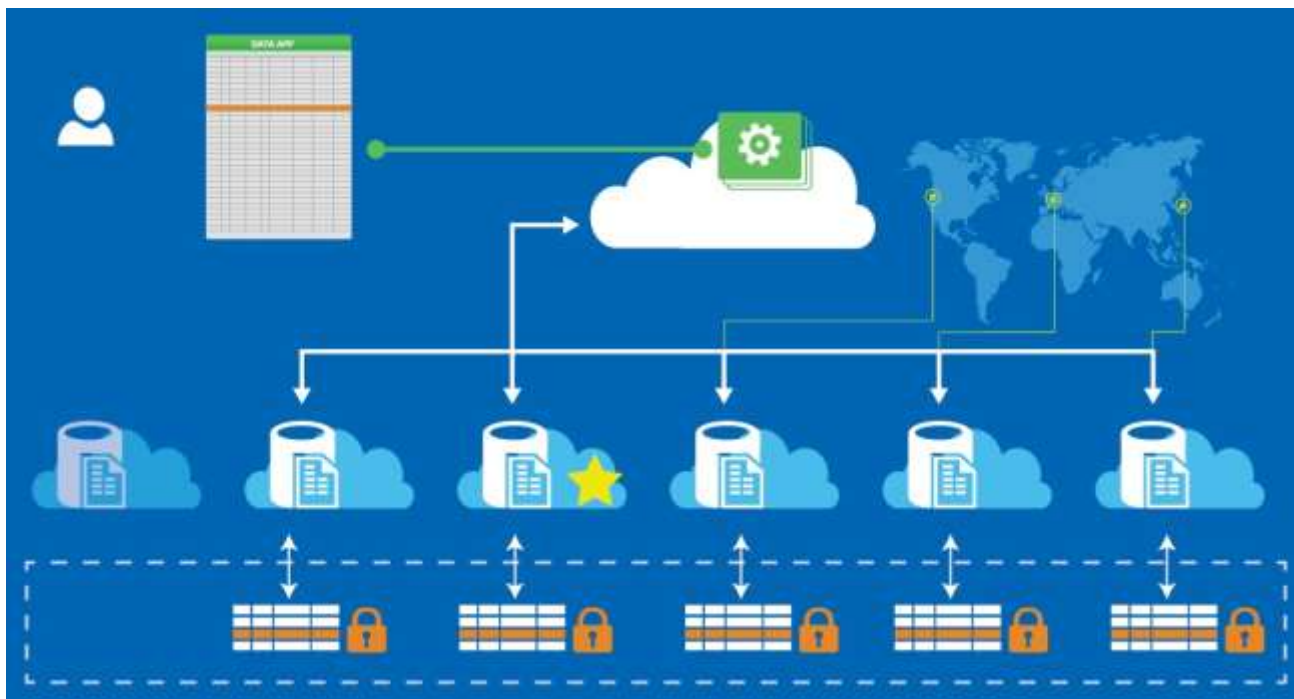


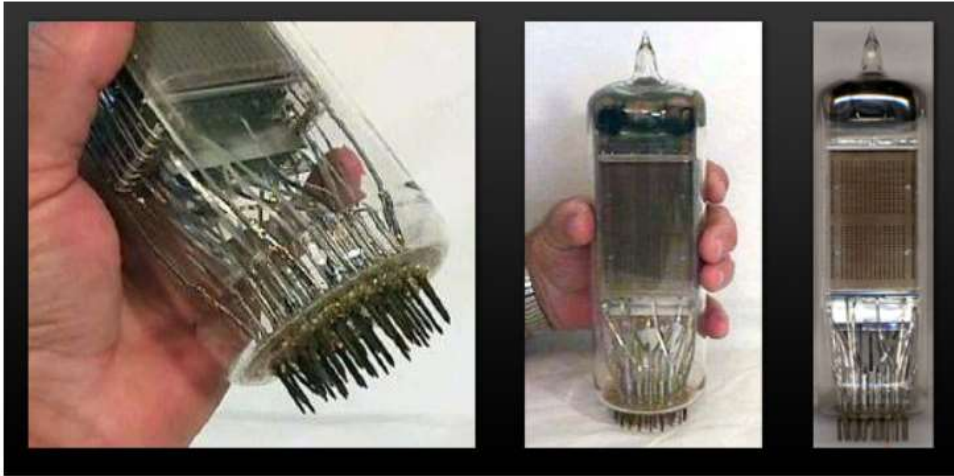


Arquitecturas de Almacenamiento y Distribución de Información

En la clase de hoy

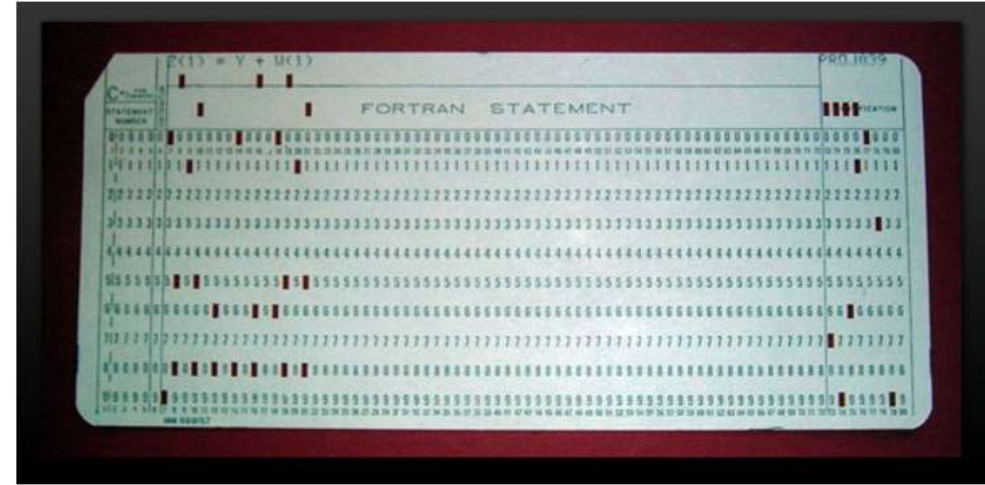
- Almacenamiento.
- Distribución.
- Arquitectura de Distribución.
- Consistencia de la información.
- Base de Datos documental.

Evolución del almacenamiento

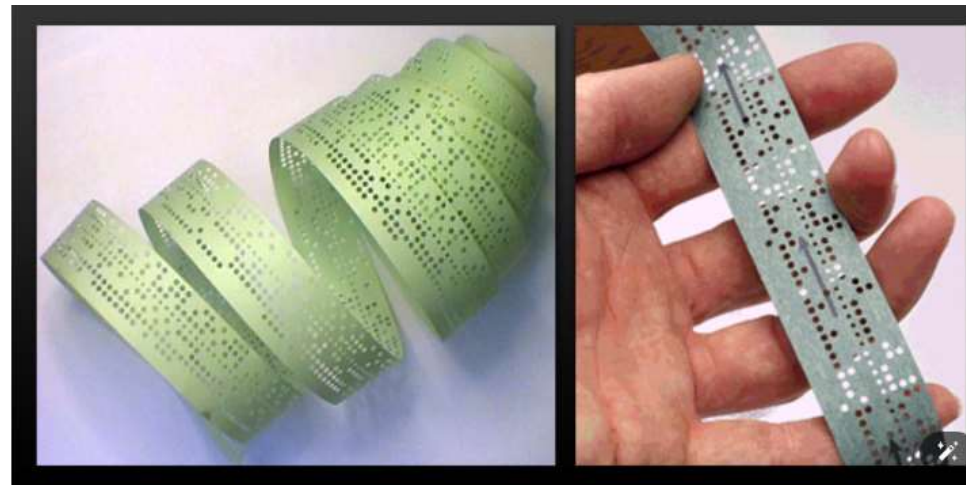


Tubo Selectron (1946) de 256 a 4096 bits

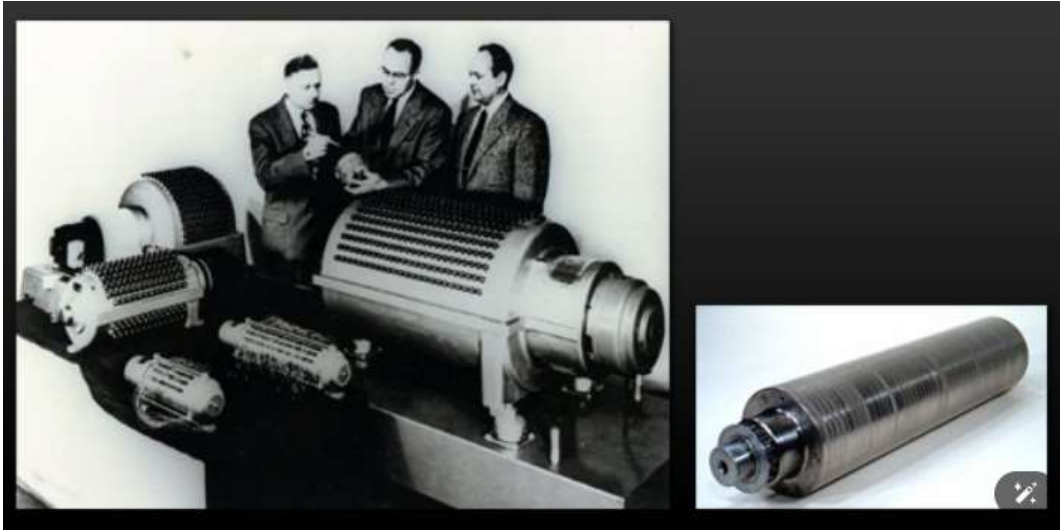
Tarjeta Perforada IBM (1948) de 72 bytes



Cinta Perforada (años '50)
de 8 bits por columna

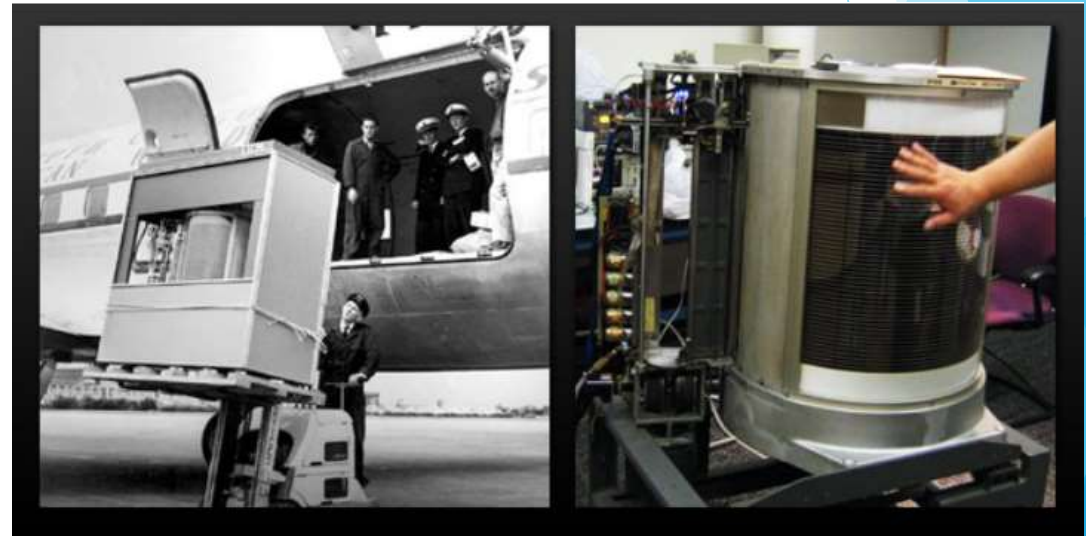


Evolución del almacenamiento



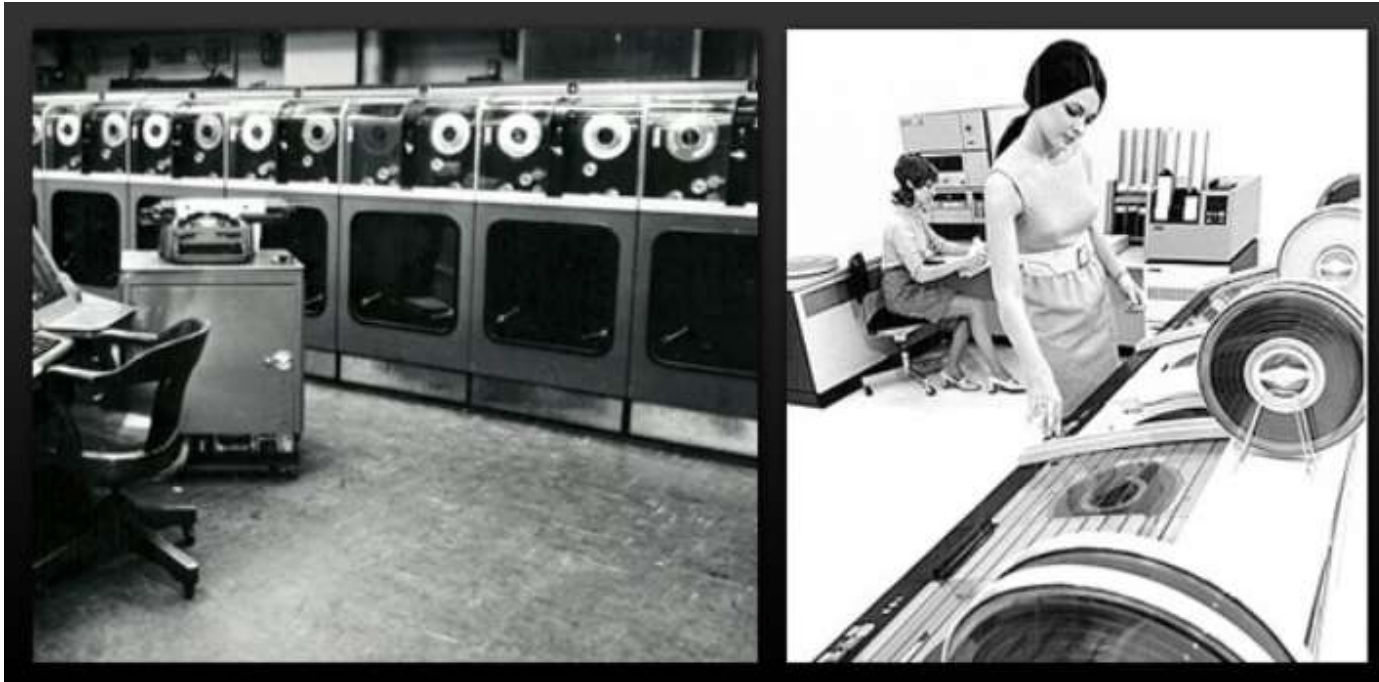
Memoria de Tambor Magnético (1932). Utilizada entre 1950 y 1960 como memoria de trabajo principal. Capacidad 10 kB.

IBM 350 Disk File (1956). Primer disco formado por 50 platos de 24" con caso 5MB

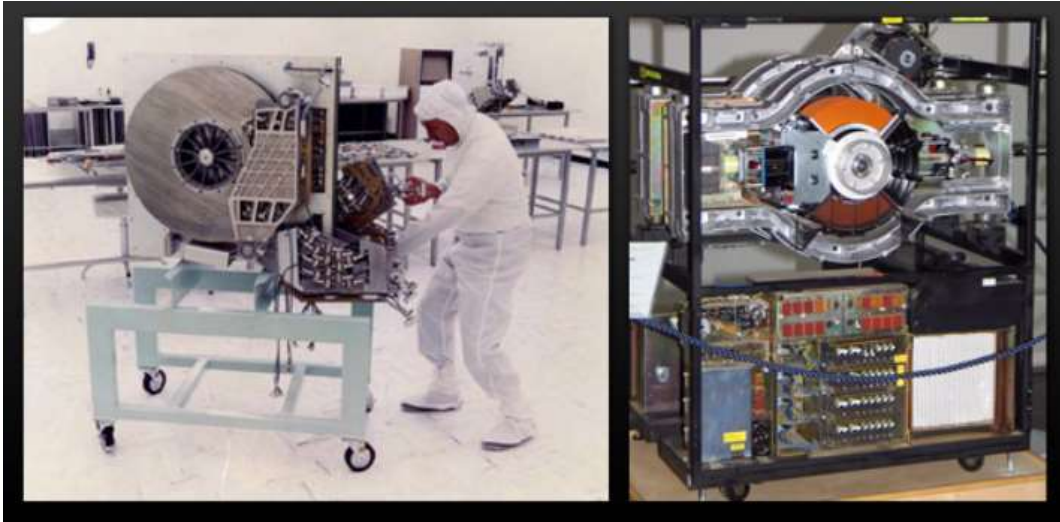


Evolución del almacenamiento

Cinta Magnética (1951). Era el principal dispositivo de E/S Las cintas eran metálicas, de 1200 pies.



Evolución del almacenamiento



IBM 3380 (1980) podía almacenar 2,52 GB, pesaba 250 kg y su precio oscilaba entre 81.000 y 142.400 dólares.

Disco Flexible (1971) tenía una capacidad de 79,7 kB y era de solo lectura. En 1972 apareció la versión de lectura y escritura.



Evolución del almacenamiento

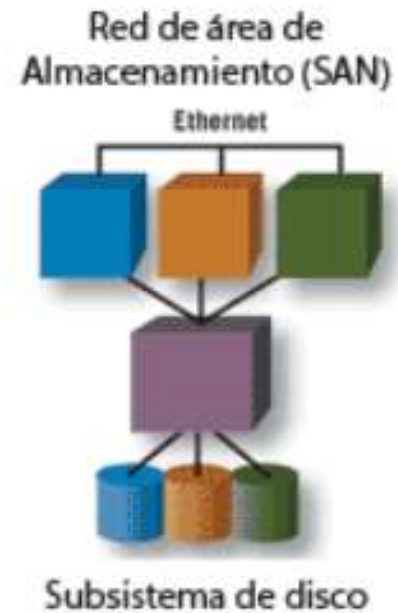
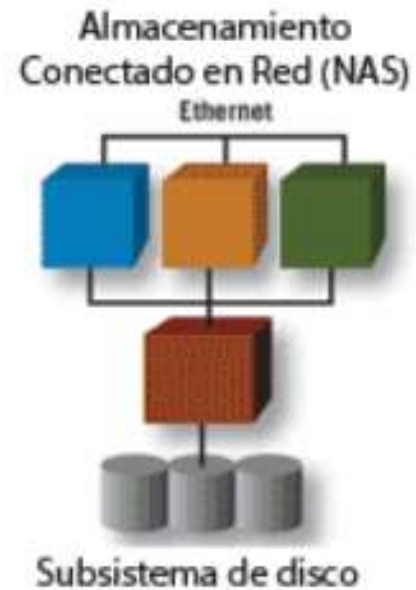
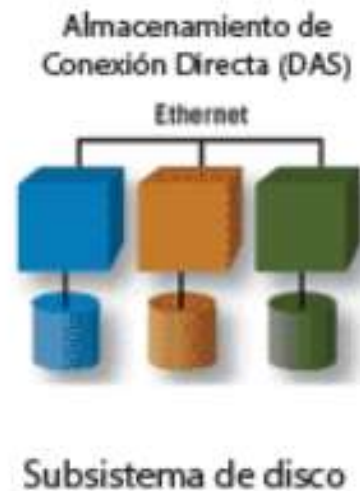


Discos Duros (1970). En los '80 se masificaron en las computadoras personales.

Disco Estado Solido (1991). Mejor performance que HDD, capacidad de sobreescritura limitada



Arquitecturas de almacenamiento



Arquitecturas de almacenamiento

- DAS (Direct Attached Storage) Almacenamiento de Conexión Directa.
- NAS (Network Attached Storage) Almacenamiento Conectado en Red.
- SAN (Storage Area Network) Red de área de Almacenamiento),



Distribución de la información.

Distribución de la información.

Concepto de Base de Datos Distribuida, presente desde fines de los 60.

Retoma a fines de los años 90.

- Objetivo: Bases de Datos que administran grandes volúmenes de información localizando datos.
- Alta interacción, múltiples usuarios concurrentes y operación continua.

Descripción

- Mezcla de tecnologías
- Tecnología de base de datos.
- Tecnología de redes y comunicación de datos.
- Las organizaciones ya no están interesadas en bases de datos centralizadas gigantes.
- Descentralización de procesos (a nivel del sistema).
- Integración de la información dentro de Bases de Datos ubicadas geográficamente en distintos lugares.

Características de una Base de Datos Distribuida

Cada nodo proporciona un entorno de ejecución tanto local como global.

Posee un esquema lógico global único:

Datos integrados permiten la recuperación y la actualización.

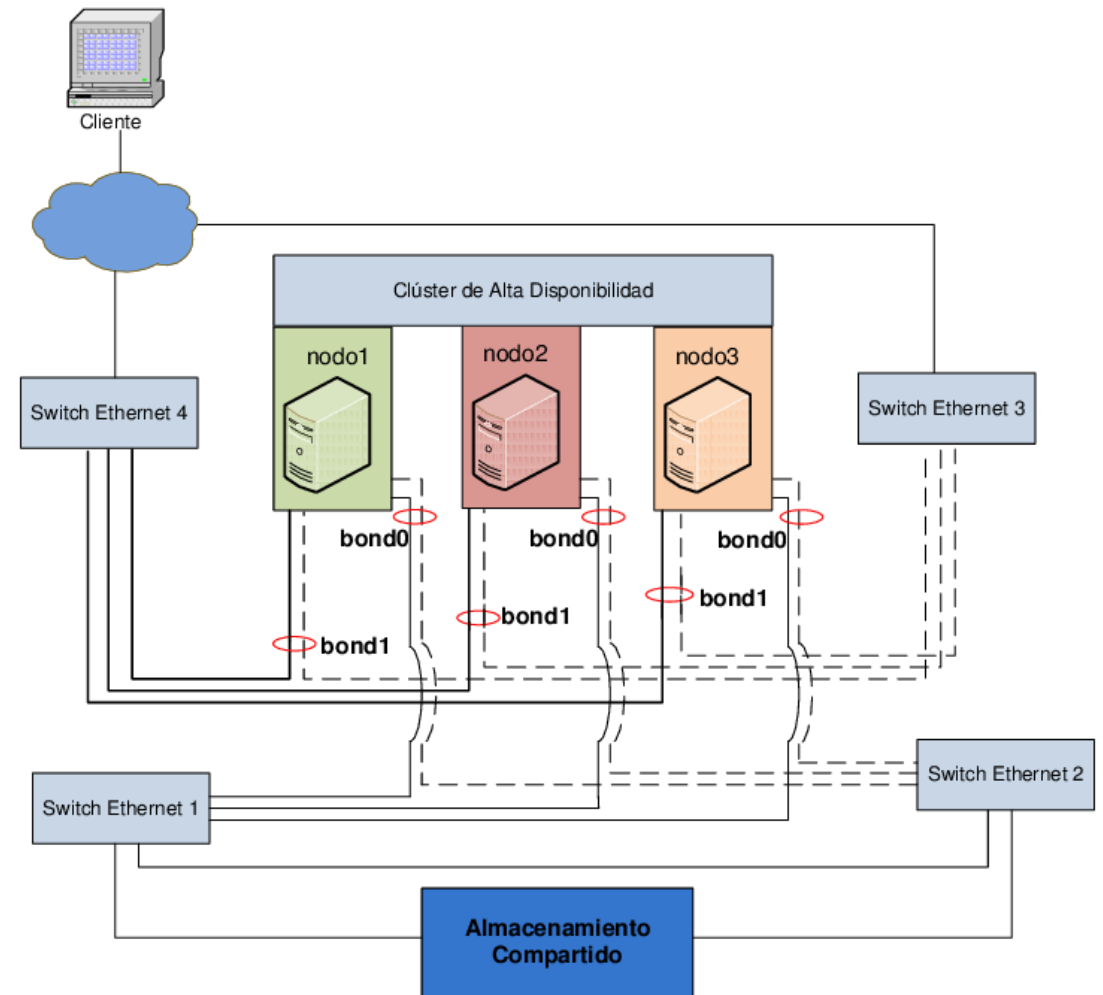
Única operación para acceder a datos replicados.

Transacciones Locales

Transacciones Globales

Cluster

Grupo de servidores independientes interconectados a través de una red dedicada que trabajan como un único recurso de procesamiento



Cluster

- Ventajas.
- Distribuyen la carga entre los servidores interconectados
- Mejoran la disponibilidad de los usuarios
- Mejoran la performance total
- Mejoran la tolerancia a fallas
- Un servidor caído es inmediatamente inactivado y los usuarios redirigidos al resto.

Tipos de Escalamiento

ESCALABILIDAD VERTICAL



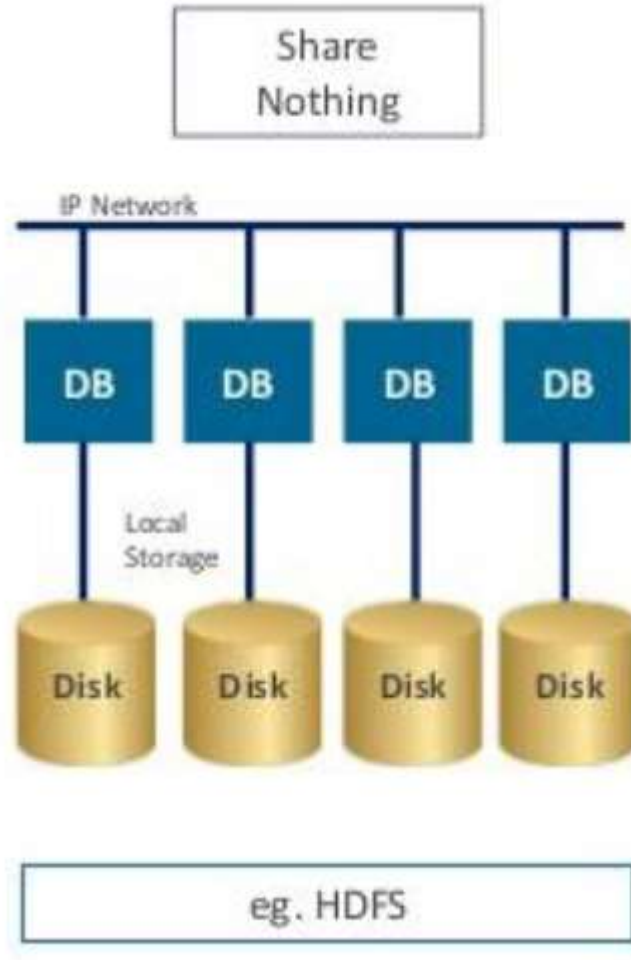
ESCALABILIDAD HORIZONTAL



Escalamiento Horizontal Permite

- Replicación.
 - Replicación Maestro-Esclavo(Master/Slave)
 - Replicación Esclavo-Maestro-Esclavo (Slave-Master-Slave)
 - Replicación entre pares(PeerToPeer)
- Particionamiento.
- Híbrida (Replicación y Particionamiento).

Arquitectura Share Nothing

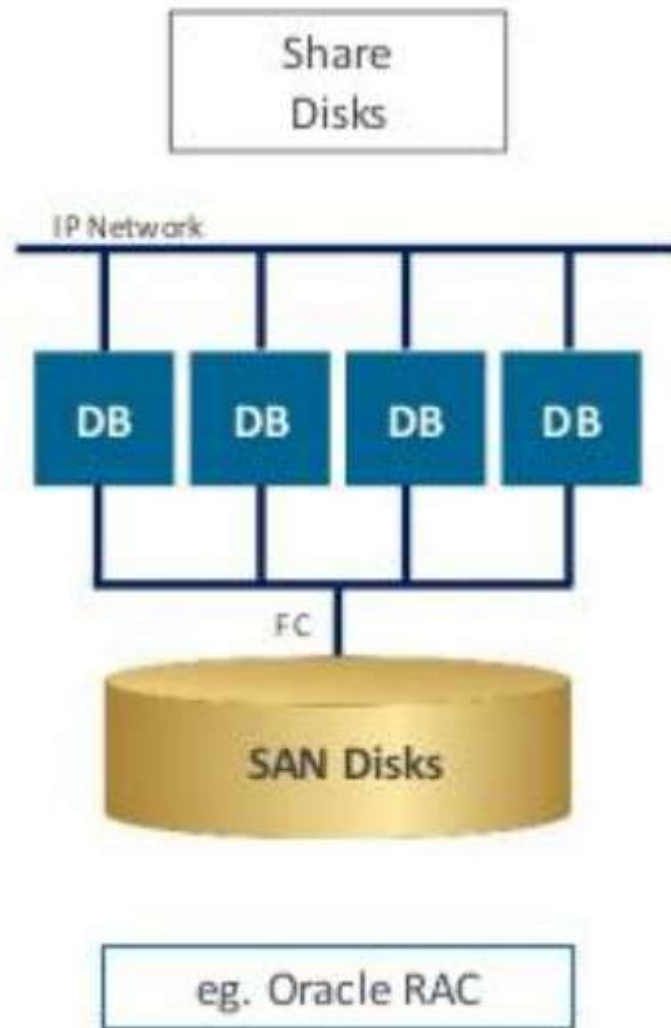


- Arquitectura distribuida, cada nodo es independiente y autosuficiente.
- Alta escalabilidad.
- Replicación de Datos.
- No SPOF (*Single Point Of Failure*).

Arquitectura Shared Nothing

- Crecimiento casi sin límite.
- No hay un cuello de botella en el sistema.
- No existe un único punto de contención (integración manual o automática).
- Tecnologías: Grids , HADOOP

Arquitectura Shared Disks



- Cada nodo tiene acceso exclusivo a su propia memoria.
- Todos los nodos pueden acceder al almacenamiento.

Arquitectura Shared Disks

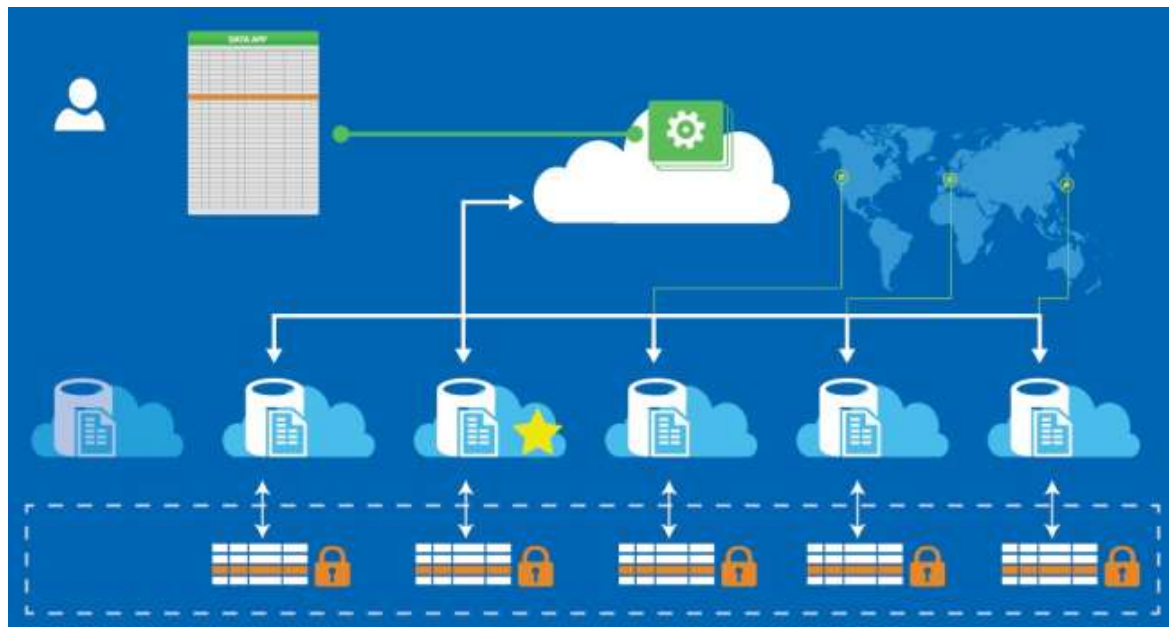
- Cada nodo tiene su propia copia del sistema operativo (Ej. mainframes)
- Viable para aplicaciones y servicios con un modesto acceso a datos.
- Aplicaciones que son difíciles de particionar.

Computación Distribuida

...Conjunto de elementos de procesamiento (no necesariamente homogéneos) interconectados por una red que cooperan en forma coordinada en la ejecución de tareas...

- Divide problemas grandes e inmanejables en piezas más pequeñas que se resuelven en forma coordinada.
- Se aprovecha mejor la potencia del procesador en la resolución de tareas complejas.
- Cada elemento de procesamiento se puede gestionar en forma independiente en el desarrollo de tareas locales (autonomía).

Consistencia de datos



Consistencia

- Desafío: gestionar la consistencia de los datos.
- Gestionar la consistencia: mantener la información uniforme a lo largo de una red o aplicaciones.
- Implica validar de acuerdo a reglas o modelos predefinidos
- La consistencia es una de las propiedades ACID mencionadas anteriormente.

Consistencia

- Las bases de datos relacionales (RDBMS) presentan una consistencia fuerte.
- En el mundo NoSQL , aparecen términos como "Teorema CAP" y "Consistencia Eventual."
- La consistencia no es evaluada en términos absolutos, sino en base a necesidades.
- **CAP** son las siglas de Consistencia – Disponibilidad – Particionamiento (**C**onsistency-**A**vailability-**P**artition tolerance).

“C” - Consistencia

- El acceso a datos es considerado plenamente consistente cuando una actualización escrita en un nodo está inmediatamente disponible en **todos** otro nodo del cluster.
- Se consigue a través de un esquema de transacciones distribuidas.
- Demoras y lazos mortales.

D - Disponibilidad

- Garantiza respuestas para todos los requerimientos que recibe (aún con nodos caídos).
- Una base de datos con un solo nodo no es posible (SPOF Single Point of Failure)
- Para mejorar la disponibilidad los datos deben replicarse en varios nodos.

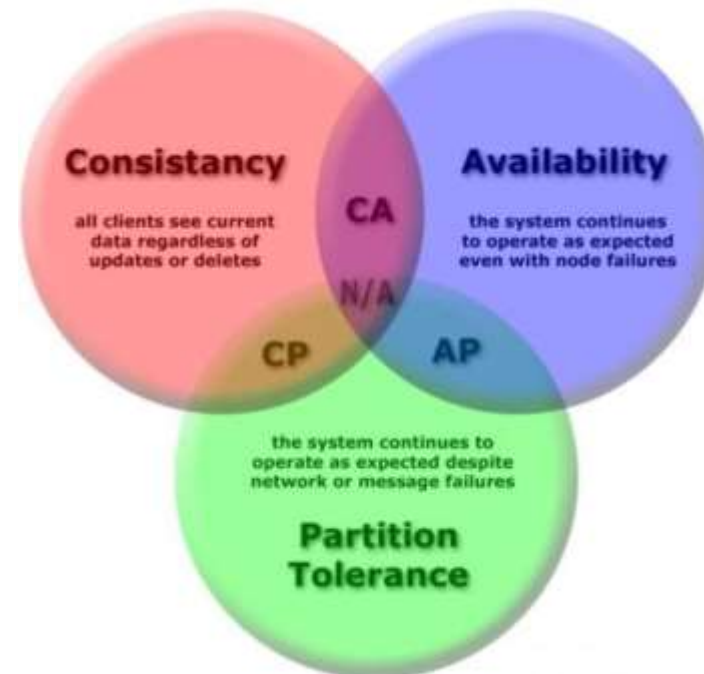
P – Tolerancia al Particionamiento.

- Los nodos pueden estar físicamente separados pero conectados.
- Partición de red: lapso en el que no puedan alcanzarse entre.
- Durante una desconexión, los nodos deberían poder leer y escribir.
- El sistema automáticamente reconcilia las actualizaciones tan pronto como cada nodo puede enlazar (escenario ideal).

Teorema CAP

- el computador científico Eric Brewer de la Universidad de California lo plantea una conjetura.
- Seth Gilbert y Nancy Lynch (2002) publican una prueba formal de la conjetura de Brewer, transformándolo en un teorema.

Es imposible para un sistema computacional distribuido, proveer simultáneamente las propiedades de consistencia, disponibilidad y capacidad de particionamiento.

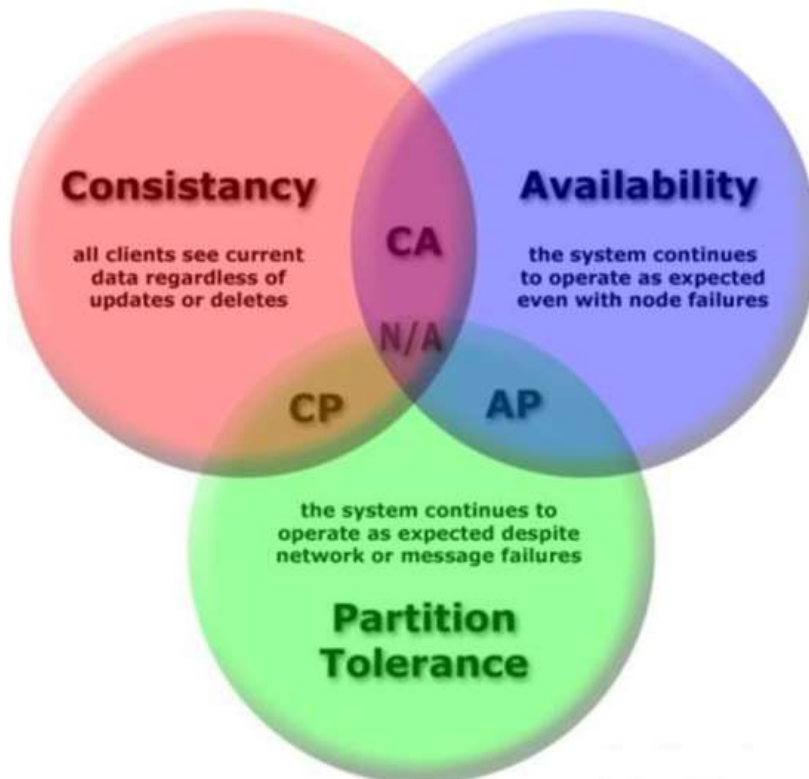


Teorema CAP

- Un sistema distribuido solo puede proveer a lo sumo dos de las tres propiedades enunciadas.
- Se deben tomar decisiones sobre cuál de los atributos sacrificar en función del objetivo deseado.
- Este enfoque fue utilizado por sistemas de bases de datos distribuidos basados en quórum que presentan algunos de los motores NoSQL como MongoDB
- Muchas de las bases de datos NoSQL pueden configurarse para cambiar el tipo de CAP utilizado (requiere reconstruir).

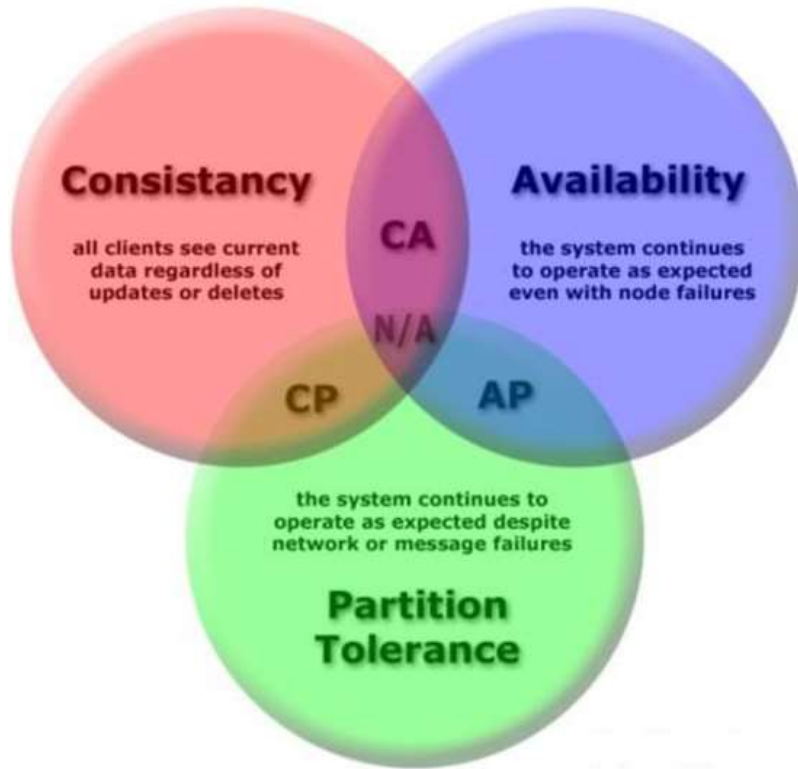
Teorema CAP

- El teorema define cuatro espacios diferentes en función de cuales propiedades se privilegian por encima del otro:



- ✓ Espacio N/A
- ✓ Espacio CA
- ✓ Espacio CP
- ✓ Espacio AP

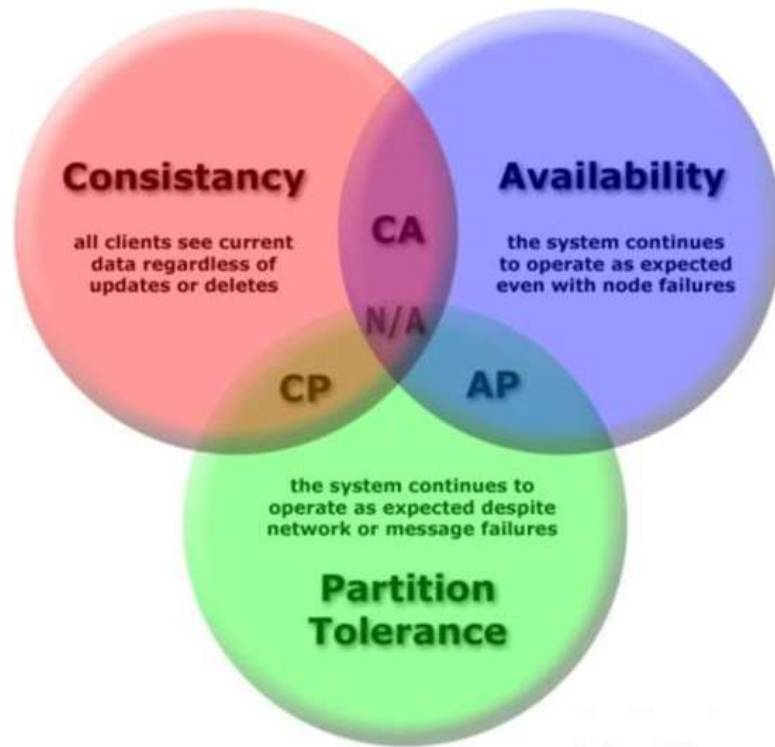
Teorema CAP



Espacio N/A.

- Este espacio es un conjunto vacío, ya que contradice al Teorema CAP.
- No existe en el estado de la tecnología actual, ningún motor de base de datos que cumpla en forma absoluta con las tres propiedades

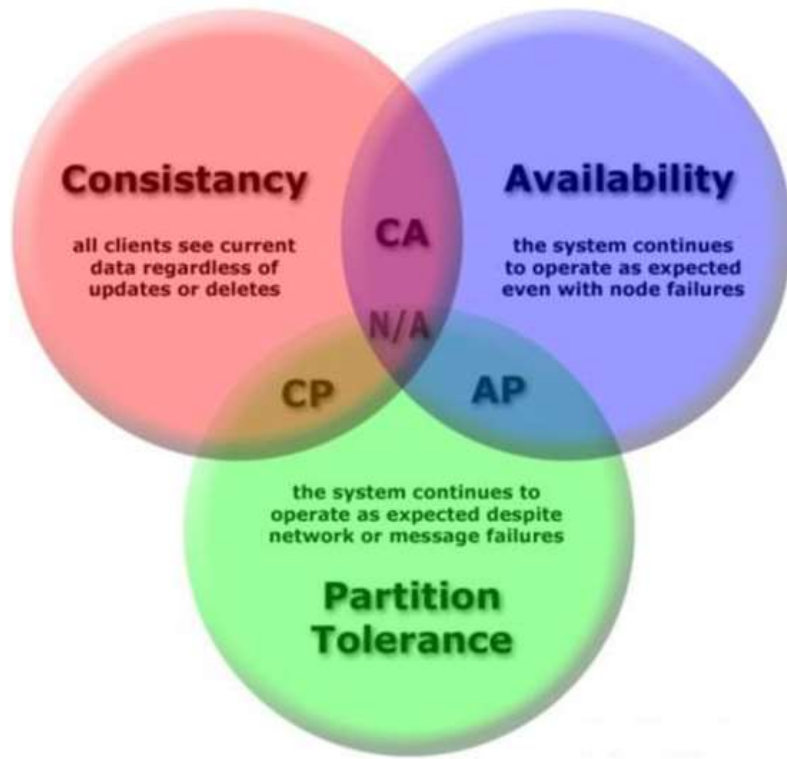
Teorema CAP



Espacio CA

- Privilegia la consistencia y la disponibilidad. Por ejemplo los RDBMS y las bases de datos de grafos.
- En este espacio no existe distribución de datos por lo que el particionamiento no es una cuestión a considerar

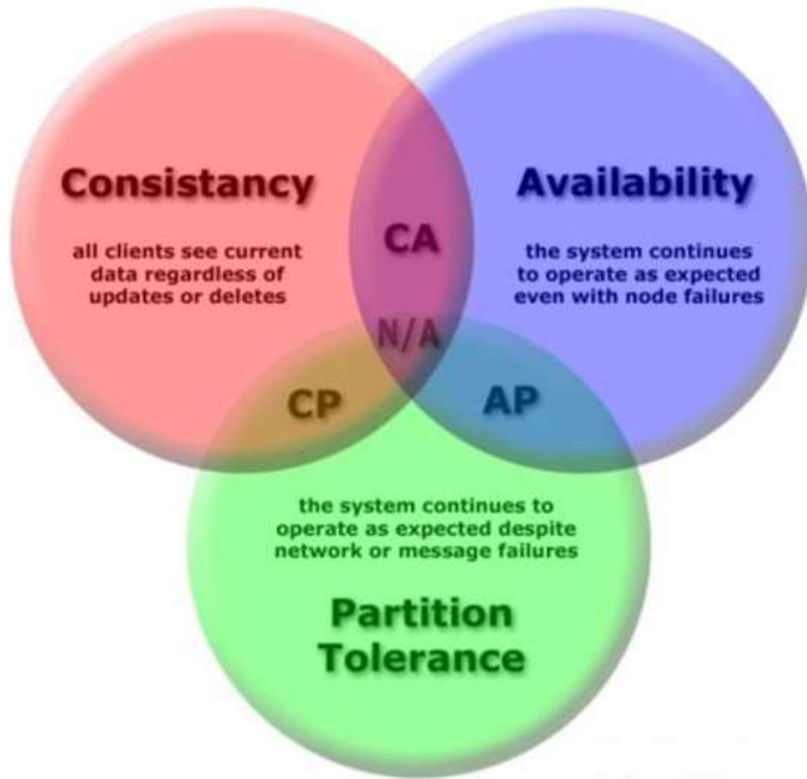
Teorema CAP



Espacio CP

- Se privilegia la tolerancia a las particiones y la consistencia.
- Algunos ofrecen consistencia fuerte en desmedro de cierto nivel de indisponibilidad (MongoDB, HBase, BigTable)
- Ante una partición de red, estas bases de datos pueden no llegar a responder a cierto tipo de consultas .

Teorema CAP



Espacio AP

- Se privilegia la disponibilidad y la tolerancia a particiones.
- Ofrecen alguna forma de consistencia relativa (Cassandra, Dynamo, CouchDB, Riak, etc.).
- Estos motores ofrecen replicación de datos, pero no garantizan consistencia entre dos o más nodos

Pero...
cuantas copias???
cuantos nodos???



Parámetros N , R y W

Cada pieza de datos se replica a una cantidad de nodos **N** en un cluster.

R es cantidad de réplicas requeridas para considerar una operación de lectura exitosa en un cluster.

W representa un valor por defecto para el cluster completo y en algunas bases de datos puede definirse para cada una operación de escritura.

***Ej. cluster con $N=5$ y un valor R de 3
(3 es la cantidad de réplicas mínimas requeridas para
considerar exitosa la operación de lectura)***

Parámetros N, R y W

- Al escribir datos (inserción o actualización) el parámetro **W** me indicará cuantas réplicas deben escribirse para que la operación sea considerada exitosa. **$W \leq N$** .
- Un valor de W alto:
 - ✓ Mayor consistencia de los datos escritos.
 - ✓ Mayor posibilidad de falla (algunos nodos pueden estar no accesibles)
 - ✓ Mayor latencia de red (hay más nodos).

Parámetros N, R y W

➤ Un valor de W bajo:

- ✓ Afectará la consistencia del conjunto de datos.
- ✓ La operación puede ser considerada exitosa al recibir confirmación de una cantidad de nodos menor.
- ✓ Involucra menos nodos, por lo que implica un tiempo menor de red.
- ✓ Las lecturas en otras réplicas pueden devolver un valor desactualizado.

Parámetros N, R y W

- Al leer datos, el parámetro R indicará cuantas réplicas deben leerse para que la operación sea considerada exitosa. $R \leq N$.
- Los primeros R nodos en devolver el mismo valor requerido forman parte del quórum de lectura.
- Cuanto más alto es el valor de R, se requiere que más nodos devuelvan el mismo valor para que la operación se considere exitosa

Parámetros N, R y W

- Un valor de R Alto
 - ✓ Ofrece un nivel de consistencia más fuerte.
 - ✓ Afecta a la performance
 - ✓ Aún con un valor de W bajo, fuerza al cluster a reconciliar piezas de datos no actualizadas que aparecen escritas con cierta inconsistencia
- Un valor de R Bajo
 - ✓ Lectura menos proclive a problemas de disponibilidad y baja la latencia de lectura

Parámetros N, R y W

➤ Consistencia Por Escrituras

- ✓ La consistencia por escrituras se determina por los valores $W=N$ y $R=1$
- ✓ Esta forma establece consistencia fuerte en la escritura con lo que la consistencia en la lectura está garantizada.
- ✓ Modelo utilizado por los RDBMS.
- ✓ Garantiza lecturas más rápidas, pero lentifica las escrituras.

Parámetros N, R y W

➤ Consistencia Por Lecturas

- ✓ La consistencia por lecturas se determina por los valores $W=1$ y $R=N$
- ✓ Implica asegurar la escritura a un solo nodo, pero leer todos ellos.
- ✓ Aunque pueden leerse algunos valores antiguos, se garantiza recuperar el valor más reciente también.

Parametros N, R y W

➤ Consistencia Eventual

- ✓ Si no existen nuevas actualizaciones que se hagan eventualmente a un objeto, todos los accesos van a devolver el valor más actualizado en último término.
- ✓ Un ejemplo son las actualizaciones a un DNS que se distribuyen en función a caches controlados por tiempo.
- ✓ Eventualmente el cliente puede ver o no la actualización, la consistencia no está garantizada, aunque existen mecanismos que la intentan maximizar

Parámetros N, R y W

Consistencia Por Quorums

- ✓ Cuanto mayor es la cantidad de nodos que se involucran, más alta es la chance de evitar inconsistencias.
- ✓ ¿Cuántos nodos deben estar involucrados en una operación para obtener cierto grado deseado de consistencia ?

Quórum de escritura: $W > N/2$

Quórum de Lectura: $R > N/2$

Parametros N, R y W

Consistencia Por Quorums

- ✓ El quórum de lectura es un poco más complicado porque depende de cuantos nodos se necesitan para confirmar una escritura.
- ✓ Si **N=3** y **W=2** requerimos 2 lecturas para asegurar que obtenemos el último dato.
- ✓ Si **N=3** y **W=1** requerimos 3 lecturas para asegurar que obtenemos el último dato.
- ✓ Se puede obtener una lectura fuertemente consistente si **$R + W > N$** .



Base de Datos Documentales



Base de Datos Documentales

- Base de datos orientada a documentos o Almacenes de Documentos.
- Los almacenes de documentos también pueden creados utilizando bases de datos de Clave / Valor.
- Permite almacenar, recuperar y administrar datos estructurados o semiestructurados.
- El término "documento" puede referirse a un documento de múltiples formatos, pero comúnmente es un archivo de XML o JSON.

Base de Datos Documentales

- El esquema es variable y proporciona mucha más flexibilidad para el modelado de documentos grandes
- Almacena cada registro y sus datos asociados en un solo documento conteniendo datos semiestructurados que pueden ser consultados mediante variadas herramientas de consulta.
- Modelado flexible de datos: Eliminan la necesidad de forzar modelos de datos relacionales para admitir nuevas aplicaciones.

Base de Datos Documentales

- Los documentos se agrupan en "colecciones", que tienen un concepto similar a una tabla relacional.
- Una base de datos de documentos proporciona un mecanismo de consulta para buscar colecciones de documentos con atributos particulares.
- Rendimiento de escritura rápido: Priorizan la disponibilidad de escritura sobre la estricta consistencia de los datos.
- Garantiza escrituras rápidas, incluso ante fallas de hardware o de la red.

Base de Datos Documentales

- Consultas rápidas y eficientes: Motores de búsqueda y funciones de indexación para mejorar el rendimiento.
- La potencia del lenguaje de consulta de una base de datos documental es un diferenciador importante entre estas bases de datos.
 - ✓ [MongoDB](#)
 - ✓ ArangoDB
 - ✓ [CouchDB](#)
 - ✓ [Amazon DocumentDB](#)



MongoDB

Versiones:

- MongoDB Community (gratuita).
- MongoDB Enterprise: (con soporte para implementación)

Estructura básica:

- Un Registro es un documento compuesto de pares de campo y valor.
- Los documentos de MongoDB son similares a los objetos JSON en su estructura.
- Los valores de los campos pueden incluir otros documentos, matrices y matrices de documentos.

Características: Seguridad

Proporciona varias características para asegurar sus implementaciones, algunas de estas son:

- Autenticación
- Autorización
- Encriptado
- TLS/SSL
- Kerberos, LDAP, Encriptado Rest
- Auditoría (solo versión empresarial)

Almacenamiento

- El motor de almacenamiento es el componente principal y es el responsable de administrar los datos.
- Existen varios motores de almacenamiento, lo que le permite elegir el más adecuado para su aplicación.
- El log es un registro que ayuda a la base de datos a recuperarse en caso de un cierre forzado. Opciones configurables para permitir un log balanceado entre rendimiento y confiabilidad.
- GridFS es un sistema de almacenamiento versátil que es adecuado para manejar archivos grandes, como los que exceden el límite de tamaño de documento de 16 MB.

Características: Índices

- Estructuras de datos especiales que almacenan una pequeña porción del conjunto de datos de la colección en una forma fácil de recorrer.
- Almacena el valor de un campo específico o conjunto de campos ordenados por el valor del campo.
- Admite coincidencias de igualdad eficientes y operaciones de consulta basadas en rangos.
- Puede devolver resultados ordenados según el orden del índice.

Características: Índices

- Similares a los índices en otros sistemas de bases de datos, define índices en el nivel de colección y admite índices en cualquier campo o subcampo de los documentos.
- Siempre se creará un índice único en el campo `_id` durante la creación de una colección.
- El índice `_id` evita que los clientes inserten dos documentos con el mismo valor para el campo `_id`.
- No se puede borrar el `_id`

Propiedades de los Índices

- Únicos; rechaza valores duplicados en el campo.
- Parciales: Solo indexan los documentos en una colección que cumple con una expresión de filtro especificada.
- Escasos: Solo contiene entradas para documentos que tengan el campo indexado.
- TTL: Son índices especiales que puede usar para eliminar automáticamente documentos de una colección después de un cierto período de tiempo.

Tipos de Índices

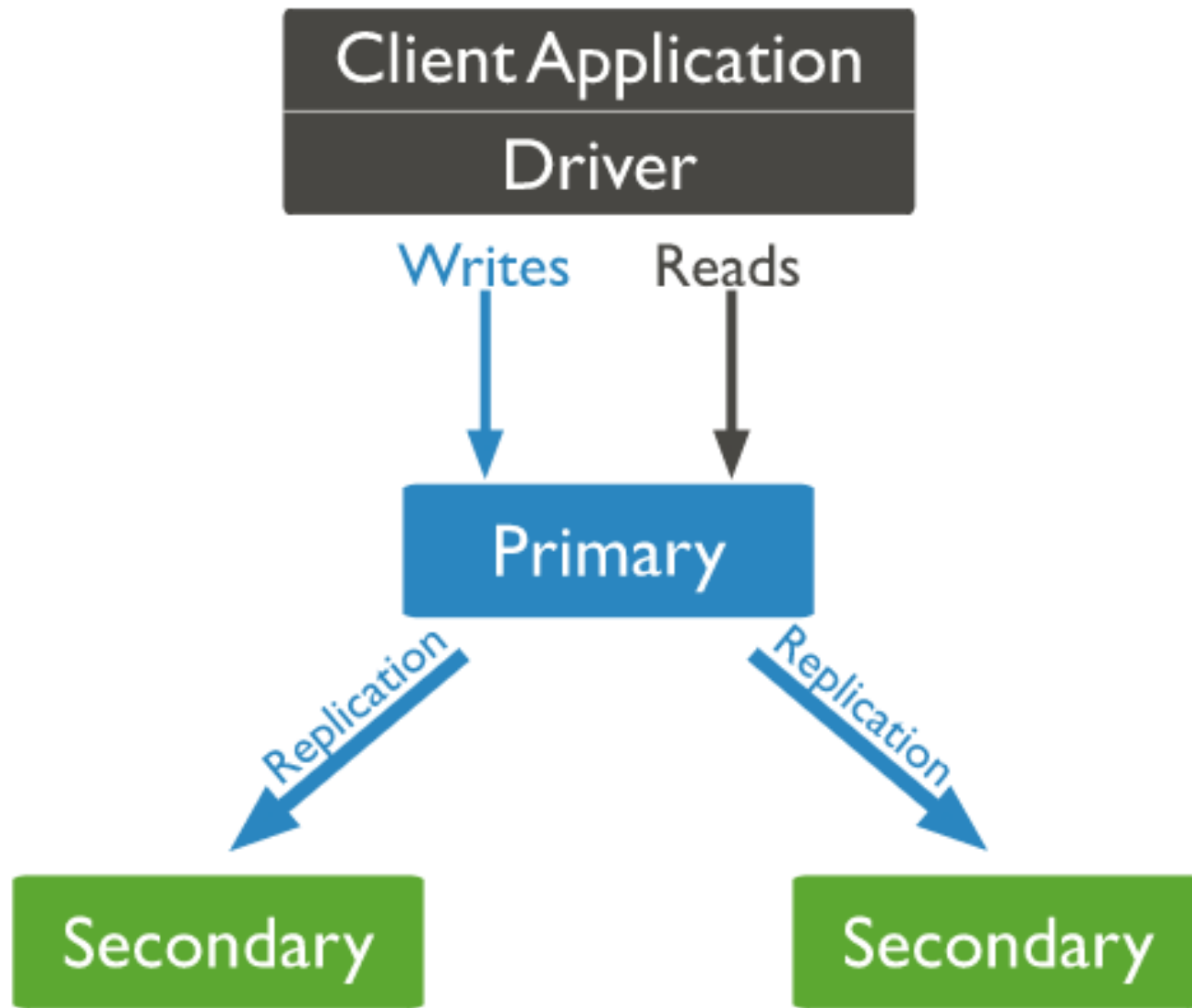
- Simple: Un campo solamente.
- Compuesto: Varios campos (importa el orden).
- Multiclave: indexa el contenido almacenado en matrices.
- Geoespacial: datos de coordenadas geoespaciales.
- Textual: admite la búsqueda de contenido de cadena en una colección.
- Hash: indexa el hash del valor de un campo. Tienen una distribución de valores más aleatoria a lo largo de su rango.

Replicación

- Un conjunto de réplicas es un grupo de instancias ***mongod*** que mantienen el mismo conjunto de datos.
- Un conjunto de réplicas contiene varios nodos que contienen datos y, opcionalmente, un nodo de árbitro.
- De los nodos que contienen datos, uno y solo un miembro se considera el nodo primario, mientras que los otros nodos se consideran nodos secundarios.

Replicación

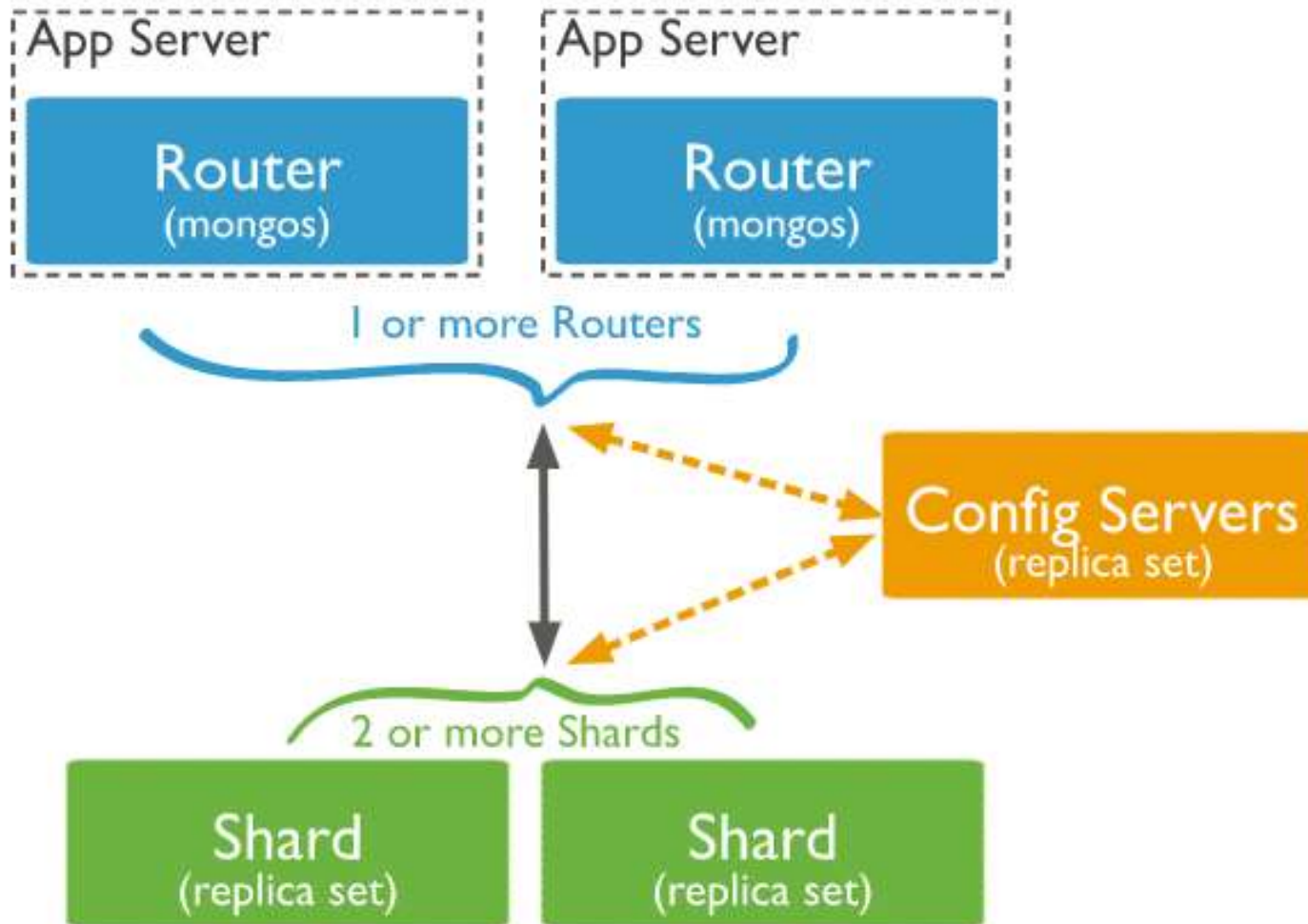
- El primario recibe todas las operaciones de escritura. Solo un primario capaz de confirmar escrituras con $\{w: \text{"mayoría"}\}$.
- El primario registra todos los cambios en sus conjuntos de datos en su registro de operaciones, es decir, oplog.
- Los conjuntos de réplicas proporcionan redundancia y alta disponibilidad y son la base de todas las implementaciones de producción.



Sharding

- Sharding es un método para distribuir datos a través de múltiples máquinas. Utiliza fragmentación para permitir implementaciones con conjuntos de datos muy grandes y operaciones de alto rendimiento.
- Acepta dos métodos de crecimiento:
 - ✓ **Escalado vertical.** (mas costoso a medida que crece)
 - ✓ **Escalado horizontal** (más barato, pero mas complejo).

Sharding (interacción de los componentes dentro de un clúster fragmentado)



Sharding - Composición

- **fragmento**: cada fragmento contiene un subconjunto de los datos fragmentados. Cada fragmento se puede implementar como un conjunto de réplicas.
- **mongos**: actúa como un enrutador de consultas, proporcionando una interfaz entre las aplicaciones del cliente y el clúster fragmentado.
- **servidores de configuración**: almacenan metadatos y ajustes de configuración para el clúster. Se deben implementar como un conjunto de réplica (CSRS - Configuration Server Replica Set).

Manejo de la Fragmentación

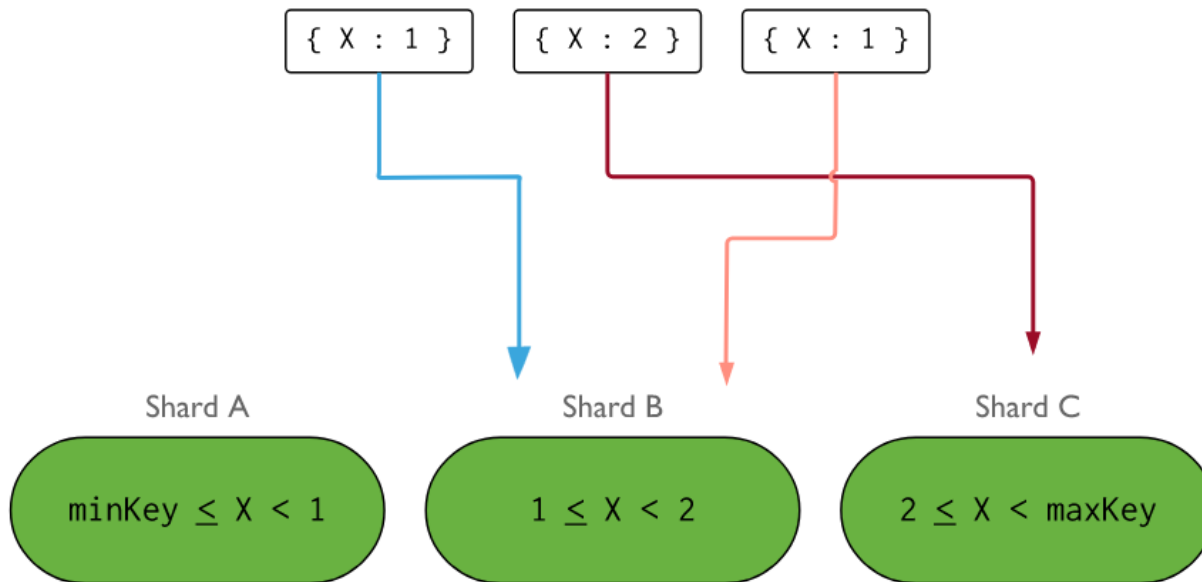
- **Shard Keys:** La clave de fragmento se utiliza para distribuir los documentos de la colección entre fragmentos.
- La clave de fragmento consiste en un campo o campos que existen en cada documento en la colección de destino.
- Hay que ser cuidadoso al elegir la clave de fragmento al fragmentar una colección ya que la elección no se puede cambiar después de fragmentar y afectará al rendimiento.
- Para fragmentar una colección no vacía, la colección debe tener un índice que comience con la clave de fragmento.

Shard Keys. Funcionamiento

- Clave para agrupar lógicamente documentos que **MongoS** se encargará de distribuir entre los distintos fragmentos, cada uno de estos grupos lógicos se le denomina **chunk**.
- El campo o campos utilizados para definir la **shard key** son los utilizados para definir el valor mínimo inclusivo y máximo exclusivo para ubicar un documento en un **chunk**.
- Todos los documentos de una **sharded collection** tienen un valor.
- Las operaciones de lectura utilizan la **shard key** para recuperar directamente el dato del **chunk** correcto.
- De no ser así, mongos tendrá que consultar todos los **chunks** para comprobar qué documentos cumplen la condición de búsqueda.

Criterio selección

Cardinalidad: número de elementos únicos que la clave puede contener, es necesaria una alta cardinalidad)

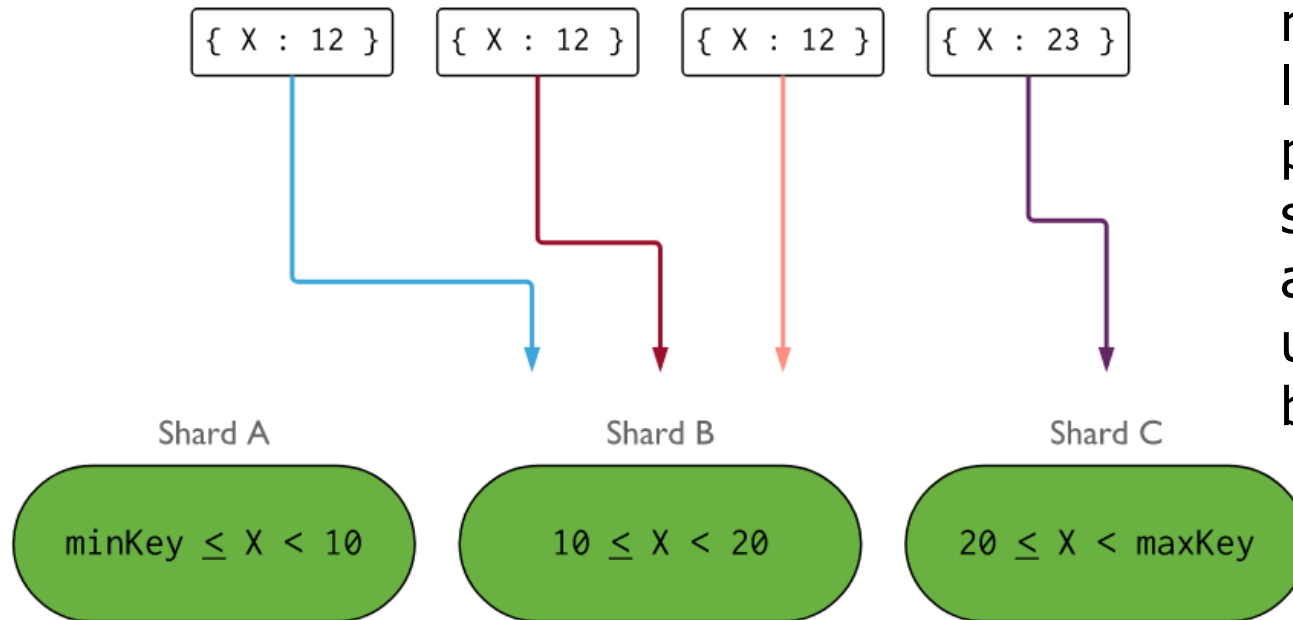


Un pequeño rango de valores, limita el número de **shards** que se pueden incluir en el **cluster**. Cada valor único sólo puede existir en un **chunk** por lo que si la cardinalidad es baja, es difícil establecer los límites de un **chunk**.

Una cardinalidad alta disminuye la posibilidad de que los valores se repitan.

Criterio selección

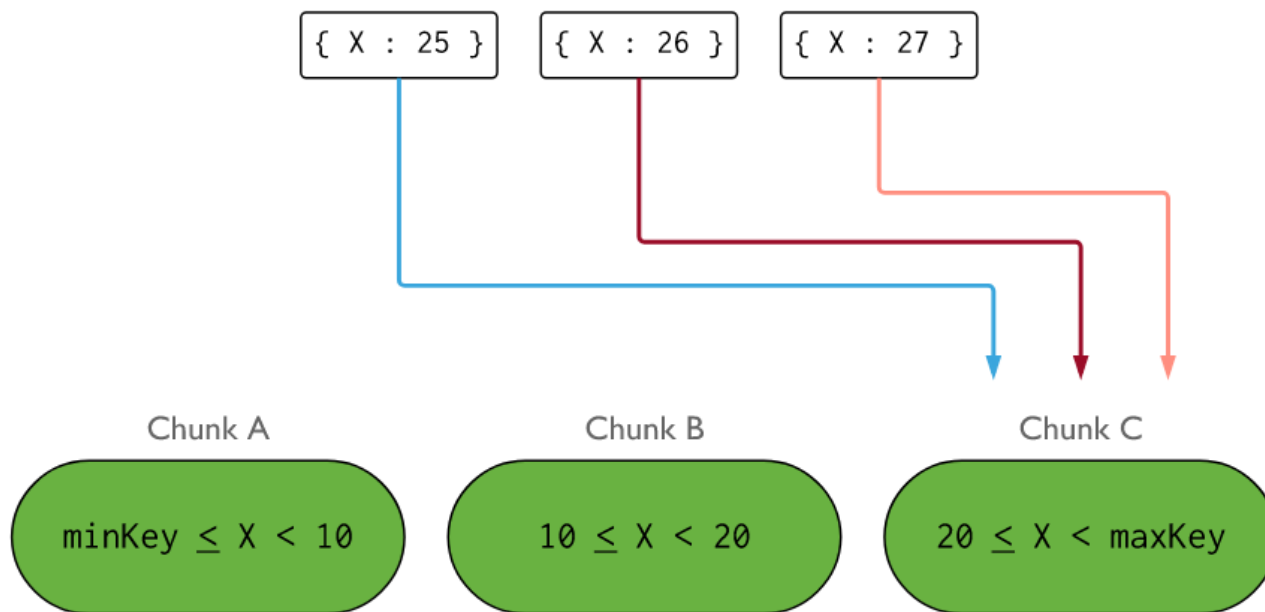
Frecuencia: las claves escogidas deben tener baja frecuencia, es decir, cada valor debería repetirse el menor número de veces posible.



Si un valor se repite muchas veces, todos los documentos irán a parar al mismo **chunk**, saturándolo y dejando al resto de **chunks** con una carga de trabajo baja.

Criterio selección

Cambios monótonos: se debe evitar que los cambios entre el valor de la clave en un documento y el valor de la clave en el siguiente documento, tengan una tasa estable y predecible.



Un cambio monótono puede ser el uso de fechas sucesivas, o claves numéricas sucesivas.

HASHED SHARD KEYS

- Calculan el valor hash de la clave y lo utilizan para decidir donde almacenar el documento, no cambian el valor sustituyéndolo por su hash sino utilizándolo en el índice correspondiente facilitando una mejor distribución de documentos entre chunks.
- Las utilizamos cuando las claves incumplen las condiciones antes mencionadas. Si aplicamos una función hash facilitamos la alta cardinalidad y baja frecuencia.
- Consideraciones:
 - Se corre el riesgo de que los documentos se distribuyan entre todos los chunks y se deba consultar todos ellos para obtener un dato concreto.
 - No podemos usar operaciones de lectura aisladas geográficamente.
 - El hash se debe calcularse sobre un único campo y que no sea un array

Documento

```
{  
  name: "sue",  
  age: 26,  
  status: "A",  
  groups: [ "news", "sports" ]  
}
```

← field: value
← field: value
← field: value
← field: value

- Los documentos (objetos) corresponden a tipos de datos nativos en muchos lenguajes de programación.
- Los documentos y arreglos integrados reducen la necesidad joins.
- Almacenamiento de documentos en colecciones.

Base de Datos y Colecciones

Los documentos se almacenan como registros de datos en colecciones y las colecciones en bases de datos. El formato de los mismo se denomina BSON.

